ColorSnap Capstone Progress Report

Name: Sijie Guo

Email: sijieg@uchicago.edu

Cnetid:sijieg

Date: April 17, 2026

1. Project Overview

Project name: ColorSnap

ColorSnap is a web app for AI-assisted personal color analysis. A user can upload a selfie, receive a seasonal color result, view a color palette, and browse beauty or fashion products that match the result.

The problem I am trying to solve is that personal color analysis is useful, but it is often expensive, hard to access, or disconnected from shopping decisions. ColorSnap makes the first step easier by giving users a quick report and practical product suggestions.

The target users are people interested in beauty, fashion, skincare, personal styling, or online shopping. A typical user might want to know which lipstick, blush, clothing colors, or metal tones fit them better before buying products.

The final capstone goal is to have an employer-ready full-stack demo that shows the complete flow: upload a photo, run analysis, view results, receive product recommendations, add items to cart, and book an expert consultation.

2. Scope and Deliverables

The original scope for this quarter was to build the main ColorSnap product experience:

- A React front end with home, analysis, result, consultation, product, cart, and payment pages.
- A backend API for image analysis, product recommendations, and health checks.
- An AI analysis flow using OpenAI when configured, with a mock mode for local testing.
- A small product catalog and rule-based recommendation logic.
- A runnable local demo with a production build.

Completed deliverables include the page structure, upload/result workflow, Express backend, analysis API, mock AI mode, OpenAI mode support, product recommendation scoring, cart persistence, and consultation booking pages.

Unfinished items include public deployment, stronger automated testing, final UI polish on older pages, authentication, analytics, and a more production-ready storage layer.

3. Progress Since Last Checkpoint

Since the last checkpoint, I focused on making the app work as one complete flow instead of separate pages.

On the front end, I connected the analysis page, result page, product detail page, cart, payment flow, consultation page, booking form, FAQ, and About page. The analysis page now supports image upload, preview, basic file validation, backend health checking, and navigation to the result screen. The result page polls the backend by analysis id and displays the completed report.

On the backend, I built the Express API under `/api/v1`. The current routes include health checks, analysis creation, analysis lookup, product lookup, and product detail lookup. Analysis jobs are created with a processing status, completed asynchronously, and stored in a local JSON file for demo use.

The app also has a working recommendation system. It scores catalog products based on season, undertone, brightness, saturation, and contrast, then returns the strongest matches. Recommended products can be added to the cart from the results page.

The current demo runs locally with the React app at `localhost:3000` and the backend at `localhost:4000`. A production build folder exists, but there is no public deployed version yet. The best demo path right now is: Home -> Analysis -> Upload photo -> Result -> Product recommendations -> Cart -> Payment or Consultation booking.

4. Technical Work

The current stack is:

- React 19 and TypeScript for the front end.
- React Router for page navigation.
- styled-components for styling.
- Express and TypeScript for the backend.
- LocalStorage for cart state and uploaded-photo preview state.
- Local JSON files for temporary backend storage.
- OpenAI Responses API support for the AI color analysis path.

The system is split into a front-end client and a backend API. The front end handles the user flow and presentation. The backend handles image parsing, analysis job creation, AI or mock analysis, result storage, and product recommendations.

One key decision was to support both mock AI and real OpenAI mode. Mock mode keeps the project easy to demo without depending on API cost, network reliability, or model latency. OpenAI mode keeps the architecture close to the real product goal.

Another decision was to keep product recommendations rule-based for now. This makes the logic easier to test and explain. A later version could use user feedback or purchase behavior, but that is outside the current capstone scope.

5. Challenges and Blockers

The biggest challenge is AI reliability. Personal color analysis depends on lighting, camera quality, background color, and skin tone visibility. I added image quality fields and retry advice, but the project still needs testing with more image types.

Another challenge is deployment. The app has a separate React client and Express backend, so I need either one host that supports both parts or a split setup with a static front end and hosted API.

The current local JSON storage is fine for a demo, but it is not enough for a real product. The app needs a database if saved reports, user accounts, order history, or analytics are added.

I would like mentor feedback on three areas: whether the analysis report is clear enough for users, what deployment option is most appropriate for this project, and how much authentication is expected for the final capstone demo.

6. Plan for Next Phase

The next phase will focus on turning the current prototype into a stronger final demo.

First, I will polish the UI so all pages feel like one product. Some newer pages already use the updated design system, while older pages still need cleanup.

Second, I will improve reliability by adding clearer loading states, error states, and tests for the main upload/result/recommendation flow.

Third, I will prepare deployment by choosing a hosting setup, configuring environment variables, testing production builds, and writing simple run instructions.

Fourth, I will decide the final scope for auth. If time allows, I will add lightweight login or saved results. If not, I will keep the demo focused on the guest user experience and list account features as future work.

7. End-Goal Readiness

ColorSnap is close to being employer-demo ready as a capstone prototype. The main product story is visible, and the core flow works locally. A reviewer can understand the idea, see the architecture, and interact with the user journey.

It is not release-ready yet. Before release, the app still needs deployment, stronger testing, real database storage, privacy handling for uploaded photos, better AI evaluation, and final bug fixing.

For the final presentation, I plan to show a live demo of the upload-to-results flow, product recommendations, cart behavior, and consultation booking. I will also show the backend API structure and explain the trade-off between mock AI mode and real OpenAI mode.

Checklist

- **Project name:** ColorSnap
- **One-sentence summary:** An AI-assisted personal color analysis app with beauty and fashion recommendations.
- **Target users:** Beauty, fashion, and styling users who want practical color guidance before buying products.
- **Quarter goal:** Build a full-stack capstone prototype with a complete demo flow.
- **This-stage goal:** Connect the front end, backend analysis flow, product recommendations, and cart.
- **Completed:** Core pages, upload flow, result polling, backend API, mock/OpenAI analysis support, product scoring, cart, booking, and production build.
- **Current demo:** Local demo through React and Express; no public deployment yet.
- **Main risks:** AI accuracy, deployment setup, storage, testing coverage, and final UI polish.
- **Next phase:** Polish UI, add tests, deploy the app, improve error states, and prepare final presentation.
- **Final showcase goal:** A live full-stack demo with upload, analysis, recommendations, cart, and consultation booking.